



APACHE KAFKA

Distributed Event Streaming — Interview Prep Guide

Architecture • Core Concepts • Delivery Semantics • CLI Commands • Python Producer/Consumer •
Spark Integration • Q&A Bank

Prepared for Big Data Engineering interviews

Table of Contents

1. What Is Apache Kafka & Why It Exists
2. Core Architecture (Brokers, Topics, Partitions, Offsets)
3. Producers & Consumers
4. Replication, Leaders & ISR
5. ZooKeeper vs. KRaft
6. Delivery Semantics & Message Ordering
7. Kafka vs. Traditional Message Queues
8. Key Producer/Consumer Configurations
9. Performance Tuning Checklist
10. Kafka + Spark Structured Streaming
11. CLI Command Cheat Sheet
12. Python Code: Producer & Consumer (fully commented)
13. Interview Q&A Bank
14. One-Page Summary — Key Takeaways

1. What Is Apache Kafka & Why It Exists

Apache Kafka is a distributed, partitioned, replicated commit log service — in practice, an event streaming platform used to publish, store, and process streams of records in real time. It was originally built at LinkedIn to solve a specific pain point: too many point-to-point pipelines between producing systems (databases, apps, services) and consuming systems (analytics, search, monitoring), which does not scale as the number of systems grows.

Kafka decouples producers from consumers by inserting a durable, high-throughput log in between. Producers write records without knowing who reads them; consumers read at their own pace, independently, and can replay history because Kafka retains data for a configurable period rather than deleting it immediately after delivery (unlike a traditional queue).

Why interviewers ask about it:

- It sits at the center of most modern data platforms (CDC pipelines, microservice communication, log aggregation, real-time analytics feeding Spark/Flink).
- It tests whether you understand distributed systems trade-offs: durability vs. latency, ordering guarantees, partitioning strategy, and failure handling.
- It connects directly to the rest of the Big Data stack you're expected to know — Hadoop/HDFS for storage, Spark Structured Streaming for processing, and Airflow for orchestration.

2. Core Architecture

Kafka's architecture is best understood as five interlocking concepts. Master these first — nearly every interview question is a variation on one of them.

Concept	Definition
Broker	A single Kafka server. A cluster is a group of brokers. Each broker hosts a subset of partitions and handles reads/writes for them.
Topic	A named, logical stream of records (e.g. 'orders', 'clickstream'). Topics are split into partitions for scalability.
Partition	An ordered, immutable, append-only log within a topic. Ordering is guaranteed only within a partition, never across the whole topic.
Offset	A monotonically increasing ID assigned to each record within a partition. Consumers track offsets to know what they've read.
Segment	Each partition is physically stored as a set of log segment files on disk; older segments are deleted or compacted per retention policy.
Producer	A client that publishes (writes) records to one or more topics, choosing (directly or via a partitioner) which partition to use.
Consumer	A client that subscribes to topics and reads records in offset order, tracking its own position (offset) per partition.
Consumer Group	A set of consumers that cooperatively read a topic; Kafka assigns each partition to exactly one consumer within the group.

TIP: Partitions are the unit of parallelism AND the unit of ordering. If you need strict ordering for a key (e.g. all events for one `customer_id`), route them to the same partition — Kafka does this by default when you provide a message key.

Mental model:

Topic 'orders' with 3 partitions looks like three independent, ordered logs (P0, P1, P2). A record with `key='customer_42'` always hashes to the same partition, so all of that customer's events stay in order. Records with different keys may land on different partitions and have no ordering guarantee relative to each other.

3. Producers & Consumers

Producer flow

- Serialize the key/value (e.g. String, Avro, JSON).
- Partitioner decides the target partition — by explicit partition number, by key hash (default, sticky per key), or round-robin if no key is given.
- Record is appended to an in-memory batch per partition, then sent to the broker once the batch is full or 'linger.ms' elapses (batching = throughput).
- Broker (the partition leader) appends it to the log and acknowledges based on the 'acks' setting (0, 1, or all).

Consumer flow

- Consumer joins a consumer group and is assigned a subset of partitions by the group coordinator (a broker-side role).
- Consumer polls the broker for new records starting at its last committed offset.
- After processing, the consumer commits the new offset (automatically or manually) so a restart resumes from the right place instead of re-reading everything or skipping data.
- If a consumer in the group dies, a 'rebalance' reassigns its partitions to the remaining consumers in the group.

TIP: Scaling rule: a partition can be read by only ONE consumer within a given group at a time. Adding more consumers than partitions in a group means the extra consumers sit idle. Partition count is therefore your upper bound on consumer parallelism for that group.

4. Replication, Leaders & ISR

Each partition can be replicated across multiple brokers for fault tolerance, controlled by the topic's replication factor (commonly 3 in production). One replica is elected the leader; all producer writes and consumer reads go through the leader. The remaining replicas are followers that continuously fetch data from the leader to stay in sync.

Term	Meaning
Leader	The replica that serves all reads/writes for a partition.
Follower	A replica that passively replicates the leader's log; can be promoted to leader if the current leader fails.
ISR (In-Sync Replicas)	The set of replicas fully caught up with the leader within 'replica.lag.time.max.ms'. Only ISR members are eligible to become the new leader without data loss.
Replication Factor	Total number of copies of each partition across the cluster (e.g. RF=3 tolerates 2 broker failures).
min.insync.replicas	Minimum ISR size required to accept a write when 'acks=all' — guards against silent data loss.

PITFALL: A very common interview trap: replication factor 3 with min.insync.replicas=1 still risks data loss on a leader failure, because 'acks=all' only waited for 1 replica to ack. Production setups typically use RF=3 and min.insync.replicas=2.

5. ZooKeeper vs. KRaft

Historically, Kafka relied on Apache ZooKeeper as an external system to store cluster metadata (broker list, topic configs, ACLs, partition leader elections). Since KIP-500, Kafka has moved to KRaft (Kafka Raft), which replaces ZooKeeper with a built-in Raft-based consensus layer run by a subset of Kafka brokers acting as controllers.

Aspect	ZooKeeper mode	KRaft mode
Metadata storage	External ZooKeeper ensemble	Internal, in Kafka itself (metadata log / <code>__cluster_metadata</code> topic)
Extra process to run	Yes — ZooKeeper cluster	No — controllers are Kafka nodes
Failover speed	Slower controller re-election	Faster, Raft-based election
Max partitions / scalability	Practically limited	Higher, since metadata bottleneck is removed
Status	Deprecated for new clusters	Default from Kafka 3.x / 4.x onward

Interview framing: be ready to explain WHY KRaft was introduced (operational simplicity, scalability of metadata, faster failover) rather than just naming it.

6. Delivery Semantics & Message Ordering

Semantic	Behavior
At-most-once	Offset committed before processing completes. If the consumer crashes mid-processing, the message is lost, never redelivered.
At-least-once	Offset committed after processing completes. If the consumer crashes after processing but before committing, the message is reprocessed on restart — duplicates possible, no loss. (Most common default.)
Exactly-once (EOS)	Achieved via idempotent producers (<code>enable.idempotence=true</code>) plus transactional writes across producer + consumer offsets, so a message is processed and reflected exactly once end-to-end.

Ordering guarantees

- Strict ordering exists only within a single partition, never across an entire topic.
- `'max.in.flight.requests.per.connection=1'` (or idempotence, which handles this internally) prevents retries from reordering records within a partition.
- If you need global ordering, you generally need a single partition — which caps throughput/parallelism, a classic trade-off to mention in interviews.

7. Kafka vs. Traditional Message Queues

Dimension	Traditional MQ (e.g. RabbitMQ)	Kafka
Data retention	Deleted once consumed / acknowledged	Retained for a configured period (or forever with compaction), regardless of consumption
Consumption model	Message removed after one consumer reads it	Multiple consumer groups can independently re-read the same data
Replay	Not supported (message is gone)	Fully supported — reset offsets and reprocess history
Ordering	Often per-queue, simpler model	Per-partition; topic-wide ordering requires a single partition
Throughput design	Optimized for transactional messaging	Optimized for high-throughput sequential disk I/O and batching
Typical use case	Task queues, RPC-style workloads (e.g. RabbitMQ)	Event streaming, log aggregation, CDC, stream processing

8. Key Producer/Consumer Configurations

Producer configs interviewers probe

Config	Values	Purpose
acks	0 / 1 / all	How many replicas must acknowledge a write before it's considered successful. 'all' is safest, slowest.
enable.idempotence	true / false	Prevents duplicate records from producer retries; required for exactly-once.
batch.size / linger.ms	bytes / ms	Controls batching: bigger batches and longer linger = higher throughput, higher latency.
compression.type	none/gzip/snappy/lz4/zstd	Trades CPU for network/disk savings; snappy/lz4 are common defaults.
retries / delivery.timeout.ms	int / ms	Controls retry behavior on transient errors before giving up.

Consumer configs interviewers probe

Config	Values	Purpose
group.id	string	Identifies the consumer group; determines partition assignment and offset tracking scope.
enable.auto.commit	true / false	Auto-commits offsets on a timer vs. manual commit for precise at-least-once control.
auto.offset.reset	earliest / latest	Where to start reading when no committed offset exists yet.
max.poll.records	int	Caps records per poll() call — tune with processing time to avoid rebalance timeouts.
session.timeout.ms / heartbeat.interval.ms	ms	Controls how quickly a dead consumer is detected and its partitions reassigned.

9. Performance Tuning Checklist

- **Partition count:** more partitions = more parallelism, but too many increases latency, open file handles, and rebalance time. Size to your target consumer parallelism, not arbitrarily high.
- **Batching:** increase 'batch.size' and 'linger.ms' on the producer for throughput-heavy workloads; lower them for latency-sensitive ones.
- **Compression:** enable 'lz4' or 'zstd' to cut network/disk usage, especially for verbose formats like JSON.
- **Consumer lag:** monitor lag (committed offset vs. log-end offset) as the primary health signal for a streaming pipeline.
- **Key distribution:** a skewed key (a 'hot partition') bottlenecks throughput even with many partitions — watch for uneven partition load.
- **Page cache:** Kafka relies heavily on the OS page cache for read performance; avoid starving brokers of memory for this purpose.

10. Kafka + Spark Structured Streaming

This is a very likely interview topic given a PySpark-focused role: Kafka is the ingestion layer, Spark Structured Streaming is the processing layer, typically landing into a medallion architecture (bronze/silver/gold).

```
# Reading a Kafka topic as a streaming DataFrame in PySpark
# (bronze layer: raw ingestion, minimal transformation)

df_bronze = (
    spark.readStream
    .format("kafka")
    .option("kafka.bootstrap.servers", "broker1:9092,broker2:9092")
    .option("subscribe", "orders") # topic name
    .option("startingOffsets", "earliest") # or "latest"
    .load()
)

# Kafka records arrive with key/value as binary – cast and parse them
from pyspark.sql.functions import col, from_json
from pyspark.sql.types import StructType, StringType, DoubleType

order_schema = (
    StructType()
    .add("order_id", StringType())
    .add("customer_id", StringType())
    .add("amount", DoubleType())
)

df_parsed = (
    df_bronze
    .selectExpr("CAST(key AS STRING)", "CAST(value AS STRING)", "timestamp")
    .withColumn("data", from_json(col("value"), order_schema))
    .select("data.*", "timestamp")
)

# Write to the silver layer as partitioned Parquet, checkpointed for
# fault-tolerant, exactly-once-per-batch semantics
query = (
    df_parsed.writeStream
    .format("parquet")
    .option("path", "/lake/silver/orders")
    .option("checkpointLocation", "/lake/checkpoints/orders")
    .partitionBy("customer_id")
    .outputMode("append")
    .start()
)
```

TIP: The checkpoint location is what gives Structured Streaming its exactly-once processing guarantee on the Spark side — it tracks which Kafka offsets have been durably written downstream, independent of Kafka's own consumer group offsets.

11. CLI Command Cheat Sheet

```
# Start ZooKeeper (only needed in legacy ZK mode, not KRaft)
bin/zookeeper-server-start.sh config/zookeeper.properties

# Start a broker
bin/kafka-server-start.sh config/server.properties

# Create a topic with 3 partitions and replication factor 3
bin/kafka-topics.sh --create --topic orders \
--bootstrap-server localhost:9092 \
--partitions 3 --replication-factor 3

# List topics
bin/kafka-topics.sh --list --bootstrap-server localhost:9092

# Describe a topic (partitions, leader, ISR)
bin/kafka-topics.sh --describe --topic orders \
--bootstrap-server localhost:9092

# Produce messages from the console (interactive)
bin/kafka-console-producer.sh --topic orders \
--bootstrap-server localhost:9092

# Consume messages from the beginning of the topic
bin/kafka-console-consumer.sh --topic orders \
--bootstrap-server localhost:9092 --from-beginning

# Check consumer group lag
bin/kafka-consumer-groups.sh --describe \
--group orders-consumer-group \
--bootstrap-server localhost:9092

# Reset offsets for a consumer group to the earliest offset
bin/kafka-consumer-groups.sh --group orders-consumer-group \
--topic orders --reset-offsets --to-earliest \
--bootstrap-server localhost:9092 --execute
```

12. Python Code: Producer & Consumer

Fully commented reference implementation using confluent-kafka (the standard production client). Written as interview-reading material — every non-trivial line explains itself.

Producer

```

# producer.py
# -----
# Publishes order events to the "orders" topic with idempotence
# and key-based partitioning enabled.
# -----
from confluent_kafka import Producer
import json

# Producer configuration:
# - bootstrap.servers: initial brokers used to discover the full cluster
# - enable.idempotence: prevents duplicate writes on internal retries
# - acks=all: wait for all in-sync replicas before acknowledging
conf = {
    "bootstrap.servers": "localhost:9092",
    "enable.idempotence": True,
    "acks": "all",
    "compression.type": "lz4", # trade CPU for smaller network payloads
    "linger.ms": 5, # small delay to allow batching
}

producer = Producer(conf)

def delivery_report(err, msg):
    # Callback invoked once per message, on success or failure.
    # This is how you detect delivery failures in an async producer.
    if err is not None:
        print(f"Delivery failed for record {msg.key():} {err}")
    else:
        print(f"Record {msg.key()} delivered to "
              f"{msg.topic()} [partition {msg.partition()}] "
              f"at offset {msg.offset()}")

def send_order(order_id: str, customer_id: str, amount: float):
    # Using customer_id as the message key guarantees all events for the
    # same customer land on the same partition, preserving per-key order.
    payload = json.dumps({
        "order_id": order_id,
        "customer_id": customer_id,
        "amount": amount,
    })
    producer.produce(
        topic="orders",
        key=customer_id,
        value=payload,
        callback=delivery_report,
    )
    # Serve delivery report callbacks without blocking (non-zero timeout
    # would block waiting for events to be processed)
    producer.poll(0)

if __name__ == "__main__":
    send_order("ORD-1001", "CUST-42", 129.99)
    # Flush blocks until all outstanding messages are delivered or fail -
    # always call this before the process exits to avoid silent data loss.
    producer.flush()

```

Consumer

```
# consumer.py
# -----
# Reads order events as part of a consumer group, with manual
# offset commits for explicit at-least-once processing control.
# -----
from confluent_kafka import Consumer
import json

conf = {
    "bootstrap.servers": "localhost:9092",
    "group.id": "orders-consumer-group",
    "auto.offset.reset": "earliest", # start from the beginning on first run
    "enable.auto.commit": False, # commit manually, after processing
}

consumer = Consumer(conf)
consumer.subscribe(["orders"])

try:
    while True:
        # poll() returns one message (or None on timeout); it also drives
        # group membership/heartbeats behind the scenes
        msg = consumer.poll(timeout=1.0)

        if msg is None:
            continue
        if msg.error():
            print(f"Consumer error: {msg.error()}")
            continue

        order = json.loads(msg.value().decode("utf-8"))

        # --- business logic goes here ---
        print(f"Processing order {order['order_id']} "
              f"for {order['customer_id']} (partition {msg.partition()}, "
              f"offset {msg.offset()})")

        # Commit only after successful processing: if the process crashes
        # before this line, the message will be re-delivered on restart
        # (at-least-once semantics – the consumer must be idempotent).
        consumer.commit(msg)

except KeyboardInterrupt:
    pass
finally:
    # Always close cleanly so the group coordinator triggers a fast,
    # graceful rebalance instead of waiting for a session timeout.
    consumer.close()
```

13. Interview Q&A Bank

Q: Why is data ordered only within a partition and not across a whole topic?

- Partitions are independent logs, each owned by one broker leader and read by one consumer per group — there's no cross-partition coordination step, which is exactly what makes Kafka horizontally scalable.
- Guaranteeing topic-wide order would require a single writer/log, eliminating parallelism.

Q: What happens when a consumer in a group fails?

- The group coordinator detects the missing heartbeat/session timeout and triggers a rebalance.
- The failed consumer's partitions are reassigned among the remaining group members.
- Processing resumes from each partition's last committed offset — so the exact re-processing behavior depends on when offsets were last committed.

Q: How does Kafka achieve high throughput?

- Sequential disk writes (append-only log) instead of random I/O.
- Zero-copy transfer from page cache to network socket (sendfile).
- Batching and compression on the producer side.
- Partitioning to spread load across brokers and disks.

Q: What's the difference between 'acks=1' and 'acks=all'?

- 'acks=1' only waits for the partition leader to write the record locally before acknowledging — a leader crash before followers replicate can lose it.
- 'acks=all' waits for all in-sync replicas (bounded by `min.insync.replicas`) to acknowledge — safer, at the cost of latency.

Q: How would you achieve exactly-once processing end-to-end?

- Enable producer idempotence (`'enable.idempotence=true'`) to dedupe retries.
- Use Kafka transactions to atomically write output records and commit consumer offsets together (read-process-write pattern).
- On the Spark side, rely on checkpointing plus idempotent sinks (e.g. upserts keyed by a natural key) rather than assuming Kafka alone guarantees it end-to-end.

Q: How do you handle a 'hot partition' (skewed key)?

- Detect it via per-partition throughput/lag metrics.
- Redesign the partition key to spread load more evenly (e.g. composite key, salting a high-volume key).
- As a mitigation, increase partition count, though that alone doesn't fix a skewed key distribution.

Q: What is log compaction and when would you use it?

- A retention policy that keeps only the latest value per key instead of deleting by time/size — the log becomes a compacted 'latest state' table.
- Used for changelog-style topics, e.g. maintaining current customer profile state for a KTable, or Kafka Connect/Streams internal topics.

Q: How does Kafka compare to Spark Structured Streaming's role in a pipeline?

- Kafka is the durable, replayable transport/buffer layer between systems.
 - Spark Structured Streaming is a processing engine that consumes from Kafka (or other sources), applies transformations, and writes to a sink — it does not replace Kafka's storage/replay role.
-

14. One-Page Summary — Key Takeaways

Topic	Key Fact
Unit of storage	Partition — an ordered, append-only, immutable log.
Unit of parallelism	Partition (producer side) / consumer within a group (consumer side).
Ordering scope	Guaranteed within a partition only.
Fault tolerance	Replication factor + ISR + leader election.
Metadata management	KRaft (modern) or ZooKeeper (legacy).
Default delivery semantics	At-least-once; exactly-once needs idempotence + transactions.
Retention model	Time/size-based, or log compaction for latest-value-per-key.
Why it scales	Sequential I/O, zero-copy transfer, batching, horizontal partitioning.
Typical Big Data role	Ingestion/transport layer feeding Spark Structured Streaming into a bronze/silver/gold medallion lake.

TIP: Before the interview: be able to draw the producer -> topic/partitions -> consumer group diagram from memory, explain acks/ISR/min.insync.replicas together as one coherent durability story, and connect Kafka to the Spark/medallion pipeline you already know.